

Analysis and Prediction of Multi-Dimensional data for Genetic Syndromes Using Machine Learning

Alexandre Silva

February 27, 2026

1 Introduction

As requested by the company, a pipeline was constructed to explore the embeddings data and extract insights from there. Many challenges were faced during the process, but we reached good results which are going to be presented here.

By means of easing the process of understanding the data and effectively creating the classification model, we employed the following tools: *Jupyter Lab* (for first analysis), *Python*, *Streamlit*, *Scikit-Learn*, *Imbalanced-Learn*, *Pandas* and *Numpy*.

At the end of the project, we created a pipeline extracting all the knowledge we gained from our *a priori* analysis done in *Jupyter Notebooks*. With this pipeline, we connected the parts in an easy-to-use dashboard with *Streamlit* which is accessible via this link <https://apollo-project.streamlit.app/>. Also, the code and experiments can be seen in this github repository <https://github.com/Dpbm/apollo-project>, as well as in the zip file within the attachments.

In this document, we aim to show our thoughts, methodologies and insights we extracted during the process. In Section 2, we're going to present how the data was parsed, transformed, analyzed and what we found at that step. In Section 3, we're going to discuss the model creation, training and evaluation. In Section 4, we're going to show the output metrics and analyze them. Finally, In Section 5 we're going to give overall insights and briefly discuss future works and improvements.

2 Data Analysis

The provided data was a pickle file, so the first step was to open it and understand its internal structure. It was known that it was a python Dictionary with 3 levels of data following the structure as shown below in Listing.1 .

```

{'syndrome_id': {
    'subject_id': {
        'image_id': [
            320-dimensional embedding
        ]
    }
}}

```

Listing 1: The pickle structure.

However, we did not know exactly how the ids nor the embedding looked like. So first of all, we employed the *python's built-in methods* for dictionaries *.values()* and *.keys()* for each *JSON* level, aiming to get a grasp of how the syndromes, subjects and images were distributed. To ease the visualization and allow us to find inconsistencies within the data, we extract the object characteristics to a *Pandas DataFrame*, this way we could manipulate the data as we wanted efficiently. For that, we used the code in Listing [2](#).

```

mapping = pd.DataFrame(columns=("syndrome", "subject", "image",
                                "image_size", "max_value", "min_value"))

index = 0
for syndrome in dataset.keys():
    subjects = dataset[syndrome].keys()
    print("-"*20)
    print(f"Looking for Syndrome: `{syndrome}`")
    print(f"Total subjects: {len(subjects)}")

    amount_of_images = set()
    for subject in subjects:
        images = dataset[syndrome][subject].keys()
        amount_of_images.add(len(images))
        total_images_per_subject = 0

        for image_id in images:
            image = dataset[syndrome][subject][image_id]
            mapping.loc[index] = {
                "syndrome":syndrome,
                "subject":subject,
                "image":image_id,
                "image_size":len(image),
                "max_value":max(image),
                "min_value":min(image)
            }
            index += 1
            total_images_per_subject += 1
    print(f"""
        Number of images for {subject} subject:
        {total_images_per_subject}""")
    print(f"Overall amount of images: {list(amount_of_images)}")
print("#"*20)
mapping.head()

```

Listing 2: *Python* code for extracting dataset characteristics into a *Pandas DataFrame*

Using this *DataFrame*, we could easily filter for *Nan* values, embeddings without the correct amount of dimensions (320), understand how many images per syndrome or either per subject the dataset has, etc. Another interesting characteristic we extracted was the min and max values for each embedding. We did it aiming to see if there were any data that could be too far away in some dimension or either too close, giving us the ability to plot the distribution later.

Now for the embeddings, we had to convert the dictionary data into a more

suitable format, in this case we moved the embeddings array into a *Numpy ndarray*. This new structure allowed us to apply transformations and plot it as we wished. The final shape of this condensed block of embeddings was (1116, 320), being in total 1116 images/rows in total. During the process, we also checked for invalid data, like *np.nan* and *np.inf*, getting rid of them using *np.nan_to_num*. The algorithm is in Listing 3.

```
EACH_ARRAY_SIZE=320 # as described by the problem
total_images = mapping.shape[0]

print(f"Creating array of {(total_images,EACH_ARRAY_SIZE)}")
data_arr = np.ndarray((total_images,EACH_ARRAY_SIZE),dtype=np.float32)

for i,row in mapping.iterrows():
    syndrome,subject,image_i = row["syndrome"],row["subject"],row["image"]
    image = dataset[syndrome][subject][image_i]
    data_arr[i] = np.nan_to_num(image)
```

Listing 3: *Python* the creation of a ndarray containing all images in sequence.

This size of it was measured and it consumed $\approx 1Mb$.

Notice that we ensured that the generated array was matching the indexes in the DataFrame by iterating over rows and using the row index as the array index.

2.1 DataFrame analysis

Using *Pandas built-in methods*, we started checking for incongruent points of data. Running the *.describe()* method, we were allowed to look for numerical incongruence within the data, but none was found as show at Table 1.

	image size	max value	min value
count	1116.0	1116.000000	1116.000000
mean	320.0	3.514369	-3.549105
std	0.0	0.632139	0.643378
min	320.0	1.807156	-7.986729
25%	320.0	3.064203	-3.889321
50%	320.0	3.433821	-3.481856
75%	320.0	3.898697	-3.111340
max	320.0	7.798529	-1.769748

Table 1: *Pandas .describe()* output for mapping DataFrame.

Image size had all instances with exactly 320 dimensions, giving *std* = 0.0. Something similar happened for min and max value, the overall deviation was

not alarming and in general the data has a solid proportion of values. It's likely that outliers may appear, but nothing that could implicate in future issues.

We also collected some primary information about the amount of syndromes, subjects and images shown in Listing.4.

```
print("--Overall statistics--")
print(f"Number of syndromes: {len(mapping.syndrome.unique())}")
print(f"Number of subjects: {len(mapping.subject.unique())}")
print(f"Number of images: {len(mapping)}")

"""
Output
-----

--Over all statistics--
Number of syndromes: 10
Number of subjects: 941
Number of images: 1116

"""
```

Listing 4: Overall statistics for number of syndromes, subjects and images in mapping DataFrame.

At this point, we started implementing the dashboard using *Streamlit* and also extracted a few more statistics from it Listing.5.

```

def get_overall_statistics(df:pd.DataFrame) -> Statistics:
    """Get Statistics from dataframe."""

    total_images = df.shape[0]
    syndromes = df.syndrome.unique()
    total_syndromes = len(syndromes)
    total_subjects = len(df.subject.unique())

    min_max_value = df.max_value.min()
    max_max_value = df.max_value.max()
    avg_max_value = df.max_value.mean()
    median_max_value = df.max_value.median()

    min_min_value = df.min_value.min()
    max_min_value = df.min_value.max()
    avg_min_value = df.min_value.mean()
    median_min_value = df.min_value.median()

    statistics = {
        "total_images":total_images,
        "total_syndromes":total_syndromes,
        "total_subjects":total_subjects,
        "min_max_value":min_max_value,
        "max_max_value":max_max_value,
        "avg_max_value":avg_max_value,
        "median_max_value":median_max_value,
        "min_min_value":min_min_value,
        "max_min_value":max_min_value,
        "avg_min_value":avg_min_value,
        "median_min_value":median_min_value,
        "syndromes_amount_of_subjects":{},
        "syndromes_amount_of_images":{},
    }

    for syndrome in syndromes:
        amount_of_subjects = len(df[df.syndrome == syndrome].subject.unique())
        statistics["syndromes_amount_of_subjects"][syndrome] = amount_of_subjects

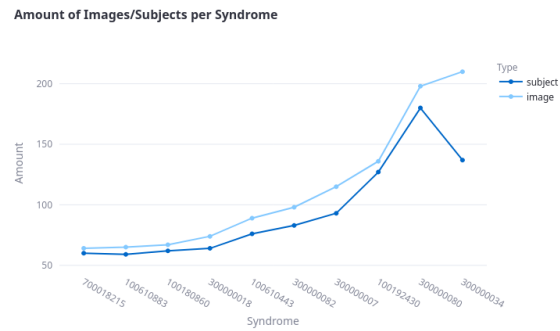
    for syndrome in syndromes:
        amount_of_images = len(df[df.syndrome == syndrome].image)
        statistics["syndromes_amount_of_images"][syndrome] = amount_of_images

    return statistics

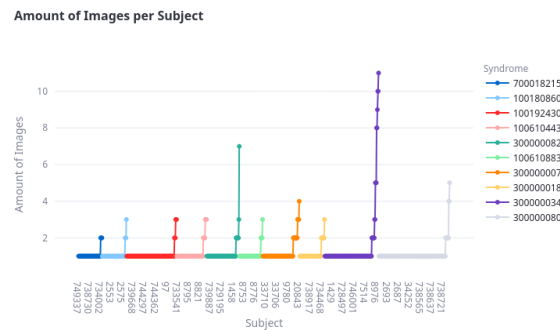
```

Listing 5: *Python* function to get overall statistics from the input DataFrame. Some more characteristics were extracted this time.

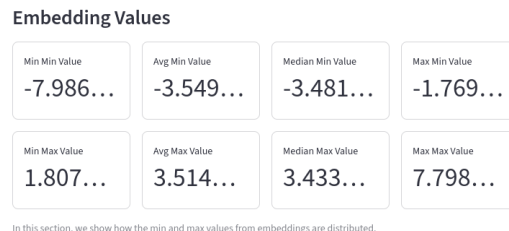
And with this new object we visualized it on the dashboard as on Figure.1.



(a) Plot showing the amount of images and subjects per syndrome.



(b) Shows the amount of images per subject.



(c) The distribution between embedding min and max values.

Figure 1: New Statistics shown on the *Streamlit* dashboard.

The plots on Figure.1 show us that the input data was imbalanced across different labels. While some had ≈ 200 images, others had ≈ 50 , being an alarming factor. To overcome this, we focused on applying *under-sampling* techniques. For this case, we tested different approaches, but the one who

best fit our needs was *RepeatedEditedNearestNeighbours*, method available in the *Imbalanced-Learn Python package*. This method uses the *KNN(k-nearest neighbors)* algorithm to clean the noisy data, prevailing the clusters with well defined bounds.

A better approach would be *data augmentation*, but we do not know exactly how the data looked like previously, so it was not suitable for our case.

An also interesting to notice fact present on the plot Figure.1a, which is also affirmed by Figure.1b, is the pair subject-image. The plot behavior follows the expected rule that, with more people having a syndrome (subject), more images we'll have. So, those syndromes with less images might be considered rare. Due to this fact, it may be the reason for the imbalanced dataset. This fact is an overall problem (when not thinking about human health) for the model, once it may be too difficult to find this sort of data in the wild at all, since the occurrences are low.

2.2 T-SNE

Since we had a 320-dimensional embedding, it is not possible to plot it right away, so we needed a different approach.

The *T-SNE* is an algorithm designed to reduce the dimensionality of data, as done by the well-known algorithm *PCA*, but preserving its local structure.

We applied this algorithm throughout our experiments, as a way to visualize the clusters. We also applied it to test the under-sampling techniques as shown at Figure.2.

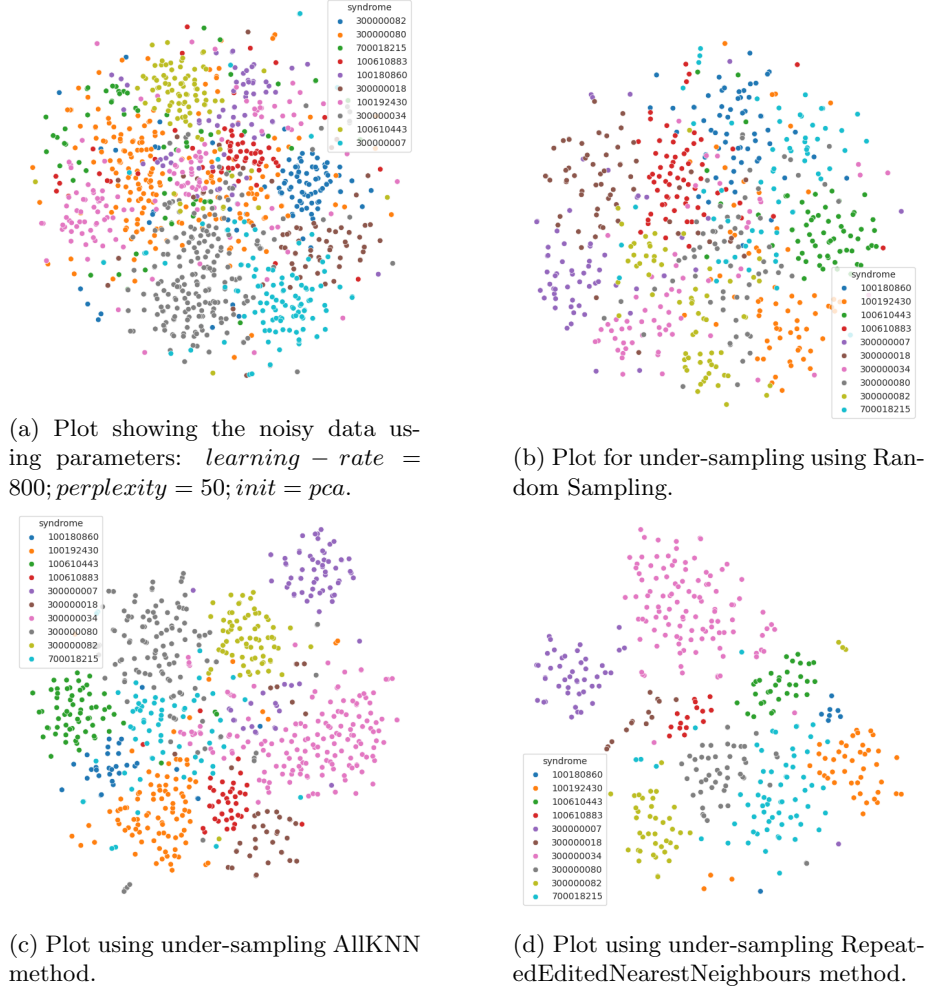


Figure 2: T-SNE plots for different techniques.

At Figure.2a we tweaked the $T-SNE$ for the dirty input data, however it is usually complicated to get good representations this way, even though some clusters may appear. On the other hand, using the under-sampling techniques, we were able to clean the data and get better visualization with default parameters as shown in Figures.[2b, 2c, 2d]. The last two were the bests, but the *RepeatedEditedNearestNeighbours* was by far the best choice, cleaning the data at a granular level.

Before each run, the data is normalized using $l2$ normalization, to ensure that the values are within the bounds $[-1, 1]$ as required by $T-SNE$.

3 Model Setup

For the model, we used *KNN* (*k*-nearest neighbors) algorithm, being one of the most common algorithms for classifying points by their proximity.

In this experiment, we tested two different metrics for *KNN*, being them *cosine* and *euclidean*. Also, different parameters of *n-neighbors* were experimented, ranging from 1 to 15. For this task, we used *GridSearchCV* from *Scikit-Learn*, to apply different combinations of these parameters and give a rank with the bests. The search used accuracy as a ranking metric and the training for each parameter was split across 10 folds for Cross-Validation.

The outcomes pointed $k = 1$ for *cosine* and $k = 6$ for *euclidean*, which are lower than expected but it is due to the dataset reduction after under-sampling. In total we had 2 models to use for predicting values.

The data used was the resampled dataset we created in Section.2, split into train and test parted 80 – 20% respectively. Then, the training batch was separated into $k = 10$ folds for Cross-Validation and the remaining test portion was used later for evaluation.

To keep the data evenly spread across folds and splits, we used *StratifiedK-Fold* and set the parameter *stratify* for *train_test_split*.

4 Performance Metrics

To ensure our model was generalizing and evolving, we used different metrics and calculated that for each fold and after evaluation. The metrics used were: accuracy, recall (micro), precision (micro), f1 (per label), ROC-AUC (OVR) and Top-K. In addition, we calculated a confusion matrix for each fold and evaluation.

For ROC-AUC per fold, we calculated and averaged that at the end, so we got, indeed, the average ROC-AUC per label across all k folds. The method used for that is discussed in References [1] and [2].

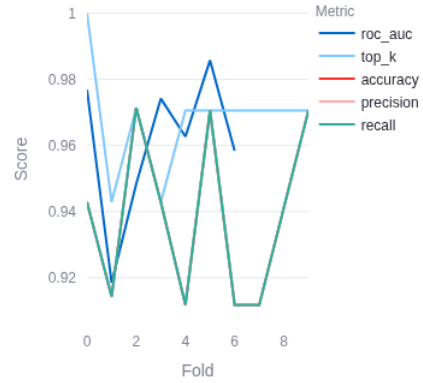
The results are shown in Figure.[3, 4] and Table.2.

Metrics for euclidean KNN model per fold



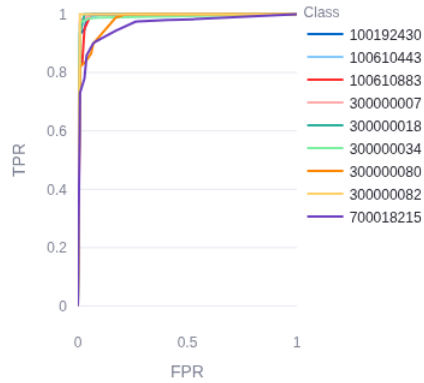
(a) Metrics for Euclidean model per fold.

Metrics for cosine KNN model per fold



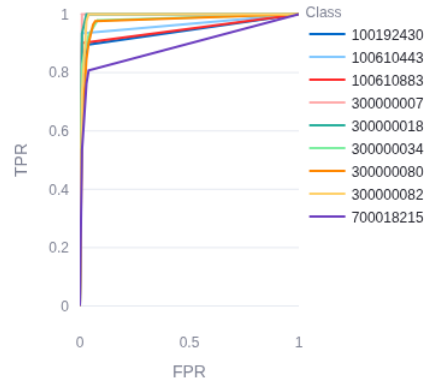
(b) Metrics for cosine model per fold.

ROC-AUC for euclidean model - mean across



(c) Mean ROC-AUC across folds for Euclidean model per syndrome.

ROC-AUC for cosine model - mean across fo



(d) Mean ROC-AUC across folds for cosine model per syndrome.

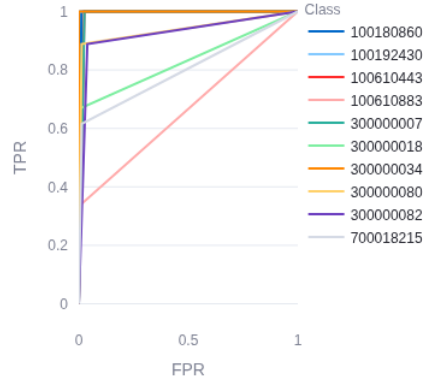
Figure 3: Output metrics per fold.

ROC-AUC model euclidean



(a) ROC-AUC for Euclidean model after evaluation.

ROC-AUC model cosine



(b) ROC-AUC for cosine model after evaluation.

Figure 4: ROC-AUC for model after evaluation.

	Euclidean	Cosine
roc_auc	0.9847	0.9132
top_k	0.9186	0.9186
accuracy	0.8140	0.8837
precision	0.8140	0.8837
recall	0.8140	0.8837

Table 2: Evaluation metrics for each model.

The results are decent given the amount of images used for training. The models reached a good precision and their ROC-AUC curve are high.

Comparing both models, Euclidean was the best, which might be due to our transformed data that had a better proximity and a straight line was the best to find neighbors than comparing angles for cosine.

One point that was observed, is that the AUC was lower for sparse classes and classes with lower density, which is understandable since the model had less chance to understand these components as clusters.

Finally, is also notable that the final performance kept faithful to what was seen during training, being always ≥ 0.8 for every metric.

5 Conclusions

Data science projects are far from easy to implement, being required creativity, technique and knowledge to develop the best pipeline as possible.

For this very project, we could get some insights with relative simple graphs and data manipulation, however since the data was limited and not much context was given, the problem became more complex to tackle. For a better outcome for machine learning projects, much is required, but knowledge about the problem is one of the chief points to succeed.

During the development, we found that the data itself was imbalanced, requiring some sort of manipulating and under-sampling. After our analysis, it stacks out that it is likely that this occurred because the lack of some diseases data, especially those that are rare. Even though they are real and occur, the model may get confused and it results in worse performance. The best option would be generating synthetic data, so it depends deeply on how the original data was alike.

Also, the final results pointed that sparse data (like the syndrome *700018215*) and smaller clusters (like the syndrome *100610883*) can lead to lower performance. Saying so, an improvement point for this dataset is increasing its diversity.

For future works we would like to state some ideas to work with. First, using a different model for comparison, like *SVM* or even creating a *Deep Neural Network*. Also, the *T-SNE* is sometimes hard to understand, since it doesn't preserve the global relationship but only local, so using *UMAP* may be a better option and a got starting point for next experiments.

References

- [1] Scikit-learn. *Receiver Operating Characteristic (ROC)*. scikit-learn. URL: https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html.
- [2] Meng Shen. *how to compute average ROC for cross validated for multi-class*. Stack Overflow, Dec. 2017. URL: <https://stackoverflow.com/questions/47876999/how-to-compute-average-roc-for-cross-validated-for-multiclass>.